

RL SYSTEM DESIGN FOR ML & AI ENGINEERS

Reinforcement Learning System Design Interview

Standard architecture patterns for scale: Deep RL, RLHF, and case studies from Netflix, Microsoft, Uber, Meta, Google, Yahoo, and Expedia.

MDPs, value methods, policy gradients, PPO, RLHF & DPO, offline RL, multi-agent systems, and 17 sourced production case studies.



Reinforcement Learning System Design Interview

Standard architecture patterns for scale: Deep RL, RLHF, and case studies from Netflix, Microsoft, Uber, Meta, Google, Yahoo, and Expedia.

Copyright © 2026 AI Engineering Insider. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except for brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

This guide is provided for educational purposes. It is not affiliated with, endorsed by, or sponsored by any company named within. Company names, interview reports, and case studies are cited to their primary sources and used for commentary and educational analysis. Compiled practice questions are labelled as such and are not represented as verbatim interview transcripts.

The information is provided “as is,” without warranty of any kind. The author shall not be liable for any loss arising from reliance on its contents.

Companion code ([rl_interview_lab](#)) is released for educational use alongside this guide.

First edition, 2026.

Web: [aiengineeringinsider.com](#) | Newsletter: [aiengineeringinsider.substack.com](#)

How to Use This Guide

Subscribe → aiengineeringinsider.substack.com/subscribe

Practitioner-first, theory dosed in exactly where an interviewer would probe it.

This guide is built the same way as the other titles in the AI Engineering Series: short conceptual explanations, worked frameworks you can reuse on a whiteboard, and an **interview** box every time a question is the kind you should expect to be asked out loud, in a room, under time pressure.

Reinforcement learning interviews differ from most ML interviews in one important way: interviewers are not usually checking whether you can reproduce an equation. They are checking whether you respect how difficult RL is in practice, whether you can reason about a system where actions influence future data, and whether you know when *not* to reach for RL at all. This guide is written with that judgment-testing structure in mind, not as a textbook substitute for Sutton and Barto.

Who this is for

This guide assumes you already know supervised deep learning (backpropagation, optimization, regularization) and are comfortable with Python and a deep learning framework. It does not assume prior RL coursework. It is written for four overlapping audiences:

RL Research Engineer / Applied Scientist

candidates interviewing at frontier labs, where RLHF, alignment, and reward modeling questions dominate the loop.

ML Engineer with an RL component

candidates whose team uses RL for one part of a larger system (recommendations, bidding, notifications) and who will be asked to reason about that system end to end.

RLHF / Alignment Engineer

candidates whose work sits at the intersection of RL and large language models, where DPO, reward over-optimization, and Constitutional AI are now standard interview topics.

Robotics / Autonomy Engineer

candidates for whom sim-to-real transfer and domain randomization are not theoretical curiosities but the actual gap between an offline-looking-great policy and a working robot.

How RL interviews differ across company archetypes

Frontier AI labs (Anthropic, OpenAI, Google DeepMind) weight RLHF, Constitutional AI, reward hacking, and alignment philosophy heavily, often combined with a systems round on training infrastructure.

Robotics and autonomy (Tesla, Waymo, Wayve) weight sim-to-real transfer, domain randomization, and safety-constrained control, usually inside a broader perception/controls loop.

Ads, recommendations, and marketplaces (Meta, Pinterest, Uber, DoorDash) weight contextual bandits, offline policy evaluation, and reward design under business constraints, more than deep algorithmic novelty.

Quant and trading weight sample efficiency, risk-adjusted reward shaping, and skepticism about RL’s reliability under non-stationary environments.

★ Key Insight

The single best signal you can give in an RL interview is judgment about when RL is the wrong tool. Candidates who treat RL as a fragile, high-leverage instrument rather than a universal hammer are consistently rated higher than candidates who can recite more algorithms.

How the chapters are organized

Parts I and II rebuild the classical and deep RL foundations the way an interviewer would actually probe them: not as a survey, but as a chain of “why was this method invented” answers. Part III covers RL for games and the sim-to-real gap, two topics that show up as standalone interview questions far more often than general RL texts suggest. Part IV is the fastest-growing segment of real interview loops: RLHF, DPO, and reward over-optimization, written to connect directly to the alignment and security-of-training-pipelines questions now common at frontier labs. Part V covers multi-agent RL at the depth interviewers actually expect. Part VI is the interview-prep payload: real, sourced case studies of RL in production at named companies, followed by a fifty-question bank split between verified real interview reports and a compiled practice set.

! Pitfall / Warning

Section A of Chapter 15 (Verified Real Interview Reports) contains only questions traceable to a named company, role, and source. Section B is compiled by the author from confirmed topic areas and industry-standard interview patterns. The two are kept deliberately separate so you always know which is which.

The companion code: `rl_interview_lab`

Every algorithm in Parts I–V is implemented in a single, runnable Python package, `rl_interview_lab`, that ships with this guide. It is a progressive library: the classical chapters share one `GridWorld` environment, and the deep-RL chapters build on `Gymnasium` and `PyTorch`, all running on a laptop CPU. Each chapter quotes and explains the exact module it corresponds to, and the console outputs printed in the book are real outputs from that code. Install it with `pip install -r requirements.txt` and run `python scripts/demo.py` to reproduce every result. Treat the code as the lab bench for the theory: read the chapter, then run the module. The full, continually-updated source for this ebook and its code companion lives at github.com/lamhotsiagian/reinforcement-learning-lab.

Book at a glance

Ch	Title	Main focus
1	MDPs, States, Actions & Rewards	The formalism; Bellman equations; state/reward design
2	Dynamic Programming	Policy/value iteration; contraction; the deadly triad
3	Monte Carlo Methods	Model-free returns; bias-variance; the n -step spectrum
4	Q-Learning & SARSA	On- vs. off-policy; TD error; overestimation
5	Deep Q-Networks	Replay, target nets, Double DQN, Rainbow
6	Policy Gradients (REINFORCE)	Policy gradient theorem; baselines; variance
7	Actor-Critic (A2C/A3C)	Advantage, the critic, GAE, sync vs. async
8	PPO	Clipped surrogate; TRPO; DDPG/TD3/SAC
9	RL for Games	AlphaZero, MCTS, self-play as curriculum
10	Sim-to-Real Transfer	Reality gap; domain randomization; system ID
11	Reward Modeling & RLHF	SFT→RM→PPO; DPO; reward over-optimization
12	Multi-Agent RL	Non-stationarity; CTDE; QMIX; self-play
13	Offline RL, Model-Based & Exploration	Distribution shift; CQL/IQL; OPE; Dreamer; RND
14	RL System Design Case Studies	17 sourced production deployments
15	Top 50 Most Asked Questions	Verified + compiled question bank
App.	Appendices & Derivations	Roadmap, notation, flashcards, reproducibility, proofs

Contents

How to Use This Guide	ii
I Foundations	1
1 MDPs, States, Actions & Rewards	2
1.1 Overview	2
1.1.1 The agent–environment loop	2
1.2 Why the Discount Factor Exists	3
1.3 Value Functions and the Bellman Equations	3
1.4 State and Action Space Design	3
1.5 The Project: a Reusable Tabular MDP	4
1.6 Comparison: MDP, POMDP, and the Bandit	5
1.7 Mock Interview Questions	5
Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation?	5
How does the discount factor γ affect agent behaviour, and what happens as $\gamma \rightarrow 1$?	6
What is the Markov property, and what do you do when your state is not Markov?	6
Distinguish V^π , Q^π , V^* , and Q^* . Which do you actually need to act?	6
A product manager wants to “use RL” for a problem with no sequential structure. How do you respond?	6
How would you formulate “reduce customer churn” as an MDP? Walk through each element.	7
Why can a small change to the reward function completely change the optimal policy, and why is that dangerous?	7
In our GridWorld the dynamics are stochastic (a 10% slip). How does that change the optimal policy versus the deterministic case?	7
What is the difference between episodic and continuing tasks, and how does each handle the return?	7
Your state space is huge and continuous. What are your options, and what do you lose with each?	8
2 Dynamic Programming	9
2.1 Overview	9
2.2 Policy Iteration vs. Value Iteration	9
2.3 Bellman Optimality as a Contraction	10
2.4 The Project: Planning on GridWorld	10
2.5 Comparison: DP Methods	11
2.6 Mock Interview Questions	12

What is the difference between policy iteration and value iteration, and when would you prefer one?	12
Prove, or at least argue, why value iteration converges.	12
What is the “deadly triad,” and why is DP immune to it?	12
Policy iteration’s inner evaluation loop is expensive. How would you speed it up without breaking convergence?	12
Why does DP need the full model, and what is the minimal change that removes that requirement?	13
Your MDP has 10^{12} states. DP is off the table, what stays useful from this chapter?	13
What is the policy improvement theorem and why does it guarantee monotone progress?	13
In our GridWorld value iteration took 15 sweeps but policy iteration only 3 updates. Reconcile that.	13
How does the discount factor interact with convergence speed and numerical conditioning in DP?	13
Asynchronous DP updates states in arbitrary order. Does that still converge, and why would you want it?	14
3 Monte Carlo Methods	15
3.1 Overview	15
3.1.1 First-visit vs. every-visit, and exploring starts	15
3.2 Architecture: the MC Control Loop	16
3.3 The n-Step Spectrum	16
3.4 The Project: MC Control on GridWorld	16
3.5 Comparison: MC vs. TD vs. DP	17
3.6 Mock Interview Questions	18
Explain the bias–variance trade-off between Monte Carlo and TD learning.	18
First-visit vs. every-visit MC, are they both correct, and which would you use?	18
Why does MC control learn Q instead of V ?	18
What problem do exploring starts solve, and what do you do when you cannot use them?	18
Walk through how off-policy MC works and why its variance can explode.	19
How does the n-step return connect MC and TD, and how do you pick n ?	19
Your MC value estimates are noisy and slow to converge in a highly stochastic environment. What concretely do you change?	19
Why is the incremental-mean update important in practice, and how does it relate to the TD step size?	19
Can MC be used with function approximation, and what breaks?	19
In our GridWorld, MC control recovered nearly the same policy as value iteration without the model. What does that demonstrate, and where would it fail?	20
4 Q-Learning & SARSA	21
4.1 Overview	21
4.1.1 The update rules, side by side	21
4.2 Architecture: Where the Two Targets Diverge	22
4.3 Convergence and Overestimation Bias	22
4.4 The Project: Q-Learning vs. SARSA on GridWorld	22
4.5 Comparison Table	24

4.6	Mock Interview Questions	24
	What is the difference between on-policy and off-policy learning, and why does importance sampling matter?	24
	Walk through SARSA vs. Q-learning on cliff-walking, where do they diverge and why?	24
	Why does Q-learning overestimate action values, and how does Double Q-learning fix it?	25
	What is the TD error, and why is it considered a biologically and algorithmically central quantity?	25
	Under what conditions does tabular Q-learning provably converge, and which condition is hardest to satisfy in practice?	25
	How do the learning rate α and exploration rate ϵ interact, and how would you schedule them?	25
	When would you choose SARSA over Q-learning in production, despite Q-learning's optimality?	26
	Expected SARSA, what is it, and where does it sit relative to SARSA and Q-learning?	26
	You set $\gamma = 0.99$ and your tabular Q-learning is unstable and slow. What do you check first?	26
	Q-learning is off-policy, which enables experience replay. Explain that link and one risk it introduces.	26
II	Deep Reinforcement Learning	28
5	Deep Q-Networks (DQN)	29
5.1	Overview	29
5.2	Architecture: the Two Stabilizers	29
5.3	The Rainbow Lineage	30
5.4	The Project: DQN with Double-DQN Targets	30
5.5	Comparison: Vanilla DQN vs. Rainbow Components	31
5.6	Mock Interview Questions	32
	Why does naive Q-learning diverge when combined with neural networks, and how does DQN fix it?	32
	What problem does Double DQN solve, and how, concretely?	32
	Explain Rainbow DQN. Which components matter most empirically?	32
	Compare ϵ -greedy, UCB, and Thompson sampling for exploration.	32
	How do you size the replay buffer and the target-sync interval, and what goes wrong at the extremes?	33
	Why use the Huber loss instead of MSE for the TD error?	33
	In a Gymnasium environment, episodes end by either termination or truncation. Why must DQN treat them differently?	33
	Your DQN's loss is decreasing but episode return is flat. Walk me through your debugging.	33
	When is DQN the wrong choice, and what would you reach for instead?	34
	Prioritized experience replay samples high-TD-error transitions more often. What bias does that introduce and how is it corrected?	34
6	Policy Gradients — REINFORCE	35

6.1	Overview	35
6.2	Deriving the Policy Gradient	35
6.3	Architecture: the REINFORCE Loop	36
6.4	The Variance Problem and the Baseline	36
6.5	The Project: REINFORCE with a Whitenened-Return Baseline	36
6.6	Comparison: Value-Based vs. Policy-Gradient	37
6.7	Mock Interview Questions	37
	Explain the policy gradient theorem in your own words, derive it if you can. . .	37
	Why does subtracting a baseline reduce variance without adding bias, and what makes a good baseline?	38
	REINFORCE is unbiased; DQN is biased. Why is REINFORCE still often harder to train?	38
	How would you scale REINFORCE to an action space of millions, like a recommender?	38
	Walk through how REINFORCE handles a continuous action space.	38
	What is the credit-assignment problem in REINFORCE, and how does the reward-to-go formulation help?	39
	Why is REINFORCE on-policy, and what does that cost you?	39
	Your REINFORCE training is extremely noisy and barely improving. What concrete levers do you pull?	39
	What role does the entropy bonus play, and what is the failure mode if you omit it?	39
	Compare REINFORCE with a learned baseline to one-step actor-critic. Are they the same thing?	40
7	Actor-Critic — A2C, A3C	41
7.1	Overview	41
7.2	Architecture: Two Heads, One Signal	42
7.3	Synchronous vs. Asynchronous	42
7.4	The Project: A2C on CartPole	42
7.5	Comparison: REINFORCE vs. A2C vs. A3C	43
7.6	Mock Interview Questions	43
	Why use an advantage function instead of raw returns in actor-critic methods? .	43
	What is the difference between A2C and A3C, and why does it matter for stability? .	43
	Why do actor-critic methods scale to continuous and large action spaces better than DQN?	44
	Walk through GAE and the role of its λ parameter.	44
	Should the actor and critic share a network body or be separate? Trade-offs? . .	44
	In the code, the advantage is detached before the actor loss. Why is that essential? .	44
	How do the actor and critic learning rates interact, and what happens if the critic is too slow?	45
	Actor-critic reintroduces bootstrapping with function approximation and (often) off-policy data. Are we back in the deadly triad?	45
	Your A2C critic loss is low but the policy is poor. What is likely happening? . .	45
	Position actor-critic between REINFORCE and PPO, what does each step add? .	45
8	PPO	47
8.1	Overview	47

8.2	From TRPO to PPO	47
8.3	The Project: a Faithful Clipped PPO	48
8.4	Continuous Control: DDPG, TD3, SAC	49
8.5	Comparison: Modern Policy-Optimization Methods	50
8.6	Mock Interview Questions	50
	What does PPO's clipped objective do mechanically, and why clip instead of a hard KL constraint?	50
	Compare DDPG, TD3, and SAC; what specific failure does each fix?	50
	Why can PPO safely run multiple epochs over the same batch when REINFORCE cannot?	50
	Walk through how you would tune PPO if it is unstable.	51
	Explain the min and clip interaction with the sign of the advantage.	51
	PPO is on-policy and SAC is off-policy. When would you pick each in a real project?	51
	Why is the importance ratio computed as $\exp(\text{new_logp} - \text{old_logp})$, and where do old_logp come from?	51
	How does PPO handle the exploration–exploitation trade-off?	52
	RLHF uses PPO to fine-tune LLMs. What changes versus CartPole, and what stays the same?	52
	A common mistake is calling SAC strictly better than TD3. Make the calibrated comparison.	52
III	RL for Games and the Real World	54
9	RL for Games	55
9.1	Overview	55
9.2	Architecture: the AlphaZero Learning Loop	55
9.3	The Project: Self-Play Tic-Tac-Toe	56
9.4	Self-Play as a Curriculum, and Its Failure Modes	56
9.5	Comparison: Game-RL Milestones	57
9.6	Mock Interview Questions	57
	Why are Atari and Go standard RL benchmarks, and what do they fail to test?	57
	Explain how AlphaZero combines learned value/policy networks with MCTS.	57
	What does MuZero add over AlphaZero, and why does it matter for non-game domains?	57
	Why is self-play described as an automatic curriculum, and when does that break?	58
	In the tic-tac-toe project, why is each player's move updated one ply late, and the loser's prior move penalized?	58
	Why was inference-time search necessary for superhuman Go when the network was already strong?	58
	How would you adapt game-RL ideas to a partially observable, real-time game like StarCraft?	58
	A candidate claims their agent “solved” a game because it beats the built-in AI. How do you stress-test that claim?	59
	Atari is model-free with unlimited data; real systems are neither. Which game-RL lessons still transfer?	59

Where does reward design go wrong in games, and how is that a preview of RLHF?	59
10 Sim-to-Real Transfer	61
10.1 Overview	61
10.2 Architecture: Train Wide, Deploy Once	62
10.3 The Project: Randomized Dynamics and a Robustness Curve	62
10.4 Comparison: Closing the Reality Gap	63
10.5 Mock Interview Questions	63
What is the sim-to-real gap, and what techniques mitigate it?	63
How exactly does domain randomization help transfer, and what is its main trade-off?	63
Design an RL-based robotics control system given the sim-to-real gap. Walk through your approach.	64
How would you measure whether a policy is actually robust, before risking real hardware?	64
When is domain randomization the wrong choice, and what would you do instead?	64
In the code, why must you recompute total_mass and polemass.length after randomizing?	65
What is the role of online fine-tuning, and what makes it risky?	65
Connect sim-to-real overfitting to reward hacking. Are they the same phenomenon?	65
Why is randomizing sensor latency and actuation delay often more important than randomizing mass?	65
Your domain-randomized policy is robust in sim across the whole range but still fails on the real robot. What now?	66
IV RLHF and Alignment	67
11 Reward Modeling & RLHF	68
11.1 Overview	68
11.2 Architecture: the Three-Stage Pipeline	68
11.3 The Project: a Reward Model and DPO, Side by Side	69
11.4 Comparison: Preference-Optimization Methods	70
11.5 Mock Interview Questions	70
Walk me through the RLHF pipeline end to end, from SFT to the final policy.	70
What is the Bradley-Terry model, and why is it used for reward modeling?	70
Why does RLHF include a KL penalty against the reference policy, and what happens if you remove it?	71
Compare PPO-based RLHF and DPO. What does DPO give up, and what does it gain?	71
What is reward over-optimization (Goodhart in RLHF), and how would you detect it during training?	71
In the project, held-out reward-model accuracy is 0.86 against an oracle ceiling of 0.89. Interpret that gap and the role of weight decay.	71
Distinguish RLAIF and Constitutional AI. Are they the same idea?	72
How is PPO actually applied to an LLM? What are the states, actions, and rewards?	72

What changed in RL for LLMs recently with GRPO-style reasoning RL, and why does it matter?	72
Tie the chapter together: are RLHF, DPO, RLAIF, Constitutional AI, and GRPO five separate techniques?	72
V Multi-Agent Reinforcement Learning	74
12 Multi-Agent RL	75
12.1 Overview	75
12.2 Architecture: Independent vs. Centralized	75
12.3 The Project: Independent Q-Learning on a Pursuit Grid	75
12.4 Comparison: MARL Approaches	76
12.5 Mock Interview Questions	77
What is CTDE, and why is it useful in multi-agent RL?	77
Why is the environment non-stationary from a single agent's perspective, and why does that matter?	77
When would multi-agent RL be the wrong choice for a problem that looks multi-agent?	77
Explain QMIX's monotonicity constraint and what it buys you.	77
How does self-play relate to general MARL, and why is it more stable?	78
In the project, two independent Q-learners coordinate fine. Does that contradict the claim that IQL is unstable?	78
What new failure modes appear in competitive (zero-sum) versus cooperative MARL?	78
How would you do credit assignment in a cooperative team with a single shared reward?	78
A team of MADDPG agents trains well in simulation but coordination collapses when one agent's policy is updated at deployment. What happened?	79
Where is genuinely multi-agent RL actually deployed, and why is it rarer than the literature suggests?	79
VI Advanced Topics	80
13 Offline RL, Model-Based RL & Exploration	81
13.1 Overview	81
13.2 Offline RL and Distributional Shift	81
13.2.1 The three families of offline algorithms	82
13.2.2 Off-policy evaluation (OPE)	82
13.3 Model-Based RL	82
13.4 Exploration Beyond ϵ -Greedy	83
13.5 Comparison: Offline RL Families	83
13.6 Mock Interview Questions	84
What is distributional shift in offline RL, and why does it break off-policy methods that work online?	84
Compare BCQ, CQL, and IQL.	84
You have one year of logged production data and cannot run live experiments yet. How do you build and validate an offline policy?	84

Why is off-policy evaluation hard, and which estimator would you reach for? . .	84
When is model-based RL worth the added complexity, and what is its central failure mode?	85
Explain MBPO's short-rollout idea. Why not roll the model out for the full episode?	85
Why does ϵ -greedy fail on Montezuma's Revenge, and how does RND help? . .	85
What is the noisy-TV problem in curiosity-driven exploration?	85
How are offline RL, model-based RL, and exploration the same problem underneath?	86
An offline policy scores far higher under your value model than the behaviour policy. Are you happy?	86
VII Interview Preparation	88
14 RL System Design Case Studies from Industry	89
14.1 The Ten-Step RL System-Design Checklist	89
Case 1 — Netflix: InTune, RL for Data-Pipeline Optimization	90
Case 2 — Wayfair: Griffin, Contextual Bandits for Notifications	91
Case 3 — Zynga: RL-Bakery and Game Personalization at Scale	91
Case 4 — DoorDash: RL Feeding a MIP for Dispatch	92
Case 5 — Alibaba: RL-to-Rank in Taobao Search	93
Case 6 — Spotify: BaRT, Bandits for Explainable Recommendations	93
Case 7 — LinkedIn: Offline RL for Mobile Notifications	94
Case 8 — Uber: Learned Value Signal for Marketplace Matching	95
Case 9 — DiDi: Standalone RL Dispatching at Scale	95
Case 10 — Pinterest: Deep RL for Ad-Ranking Utility Tuning	96
Case 11 — Booking.com: What Breaks Bandits in Production	96
Case 12 — Meta: Horizon, and Train-Complex / Deploy-Interpretable Bidding .	97
Case 13 — Google/YouTube: Top-K Off-Policy REINFORCE	98
Case 14 — Yahoo!: Real-Time Bidding by RL	98
Case 15 — Electronic Arts (SEED): RL for AAA Testing and Gameplay	99
Case 16 — Microsoft: Azure Personalizer / Xbox Top-of-Home Pilot	100
Case 17 — Expedia: Cascade Bandits for Ranking	100
14.2 Worked Senior System-Design Walkthroughs	101
14.2.1 Walkthrough 1 — RL Feed Recommendation (Meta / Google scale)	101
14.2.2 Walkthrough 2 — RLHF Pipeline for an LLM (frontier-lab scale)	102
14.3 Cross-Case Summary	103
14.4 How to Use These Case Studies in an Interview	104
14.5 Mock Interview Questions	105
Design an RL system to optimize push notifications. Walk me through it end to end.	105
When would you use a contextual bandit instead of full RL in a production recommender?	105
A stakeholder needs to explain the agent's decisions. How do you deploy RL while keeping it interpretable?	105
How would you validate an RL policy offline before risking online traffic?	105

Uber and DiDi both solve dispatch with RL but differ architecturally. Compare the choices.	106
What production realities break textbook bandit algorithms, per Booking.com?	106
How do you scale policy gradients to a recommender with millions of items?	106
When is RL the wrong tool, and how do you tell early? Use a real example.	106
Design the monitoring and reward-logging infrastructure for a deployed RL system.	107
Why do so many production systems use RL as one component feeding a classical system, rather than end-to-end?	107
15 Top 50 Most Asked Questions	108
15.1 How RL Interviews Are Weighted by Company Archetype	108
15.2 Section A — Verified Real Interview Reports	109
Explain the difference between PPO and GRPO.	109
How does Constitutional AI differ from traditional RLHF?	109
What are the limitations of Constitutional AI as an alignment approach?	110
If you were writing a constitution for a powerful AI system, what principles would you prioritize and why?	110
How would you measure whether an AI system is actually following its stated principles?	110
Explain RLHF optimization, including KL divergence and reward hacking.	110
During pretraining of a 100B parameter model, you suddenly see loss spikes. How do you debug this? Is it a data issue, learning rate problem, or hardware failure?	110
How would you prevent new privacy-preserving technologies from being exploited to make AI misuse harder to track?	110
Explain deep reinforcement learning.	110
What metrics would you use to evaluate the impact of RLHF on ChatGPT responses?	110
Be ready to discuss Q-learning, SARSA, multi-armed bandits, and DQNs.	111
Compare Q-learning (value-based) and policy gradient methods (policy-based), and discuss how each could apply to training an autonomous driving policy.	111
15.3 Section B — Compiled Interview Question Bank	111
Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation?	111
What's the difference between policy iteration and value iteration, and when would you prefer one over the other?	111
Explain the bias-variance tradeoff between Monte Carlo and TD learning.	111
What is the “deadly triad” in RL, and why does it matter for anything you'd actually deploy?	112
Walk me through SARSA vs. Q-learning with a concrete example of where they'd diverge.	112
How does the discount factor γ affect agent behavior, and what happens as $\gamma \rightarrow 1$?	112
Explain the exploration-exploitation tradeoff. Compare ϵ -greedy, UCB, and Thompson sampling.	112
What's the difference between on-policy and off-policy learning, and why does importance sampling matter here?	112
Why does naive Q-learning diverge when combined with neural networks? How does DQN fix this?	112

What problem does Double DQN solve, and how?	112
Explain the policy gradient theorem in your own words — derive it if you can.	112
Why do we use an advantage function instead of raw returns in actor-critic methods?	112
What does PPO’s clipped objective actually do, mechanically? Why clip instead of using a hard KL constraint like TRPO?	112
Compare DDPG, TD3, and SAC. What specific failure mode does each one fix relative to its predecessor?	113
What is the maximum entropy RL framework, and why does SAC use it?	113
When would you choose a model-based RL approach over model-free, and what’s the main risk in doing so?	113
What is offline RL, and why does standard off-policy correction fail when you can’t collect new data?	113
Explain Rainbow DQN. Which components matter most empirically?	113
Walk me through the RLHF pipeline end to end, from SFT to the final policy.	113
What is the Bradley-Terry model, and why is it used for reward modeling?	113
Why does RLHF include a KL penalty against the reference policy? What happens if you remove it?	114
Compare PPO-based RLHF and DPO. What does DPO give up, and what does it gain?	114
What is reward over-optimization (Goodhart’s Law in RLHF), and how would you detect it during training?	114
Describe a real or hypothetical case of reward hacking. What made the reward function exploitable?	114
What is RLAIIF, and how does it differ from standard RLHF?	114
How would you design an evaluation framework to catch RLHF-induced regressions before shipping a model update?	114
What is CTDE (centralized training, decentralized execution), and why is it useful in multi-agent RL?	114
Explain how AlphaZero combines learned value/policy networks with MCTS.	114
What is the sim-to-real gap, and what techniques mitigate it?	114
How does domain randomization help an RL policy trained in simulation transfer to the real world?	115
When would multi-agent RL be the wrong choice for a problem that looks multi-agent?	115
Why are Atari and Go considered standard RL benchmarks, and what do they fail to test for?	115
Design an RL-based bidding system for online advertising. What would you use as the reward signal, and what could go wrong?	115
How would you evaluate a new RL policy before deploying it, given you can’t safely A/B test in production?	115
Your RL agent’s training reward is climbing steadily, but real-world performance is flat or worse. Walk me through how you’d debug this.	115
Tell me about a situation where you’d recommend against using RL, even though it technically applies.	116
How would you explain a reward-shaping decision to a non-technical stakeholder who’s worried the model is “gaming the metric”?	116

Design a system to safely roll out a new RLHF-tuned model update to a fraction of production traffic. What guardrails would you build in?	116
Appendices	117
Appendix A — Study Roadmap	117
Appendix B — Notation Cheat Sheet	118
Appendix C — Rapid-Fire Flashcards	118
MDP	118
On-policy vs. off-policy	118
Bootstrapping	118
Deadly triad	118
Experience replay	119
Target network	119
Baseline (policy gradient)	119
GAE	119
Trust region	119
Domain randomization	119
Reward hacking	119
Goodhart’s Law	119
Bradley-Terry model	119
CTDE	119
Offline RL	120
Contextual bandit	120
DPO	120
GRPO	120
Self-play	120
Sim-to-real gap	120
Appendix D — Reading List, Ordered by Depth	120
Appendix E — Mapping This Guide to Company Interview Patterns	121
Appendix F — Reproducibility & Benchmarks	121
Mathematical Derivations	123
Derivation 1 — The Policy Gradient Theorem	123
Derivation 2 — Generalized Advantage Estimation	124
Derivation 3 — Value Iteration Converges (the γ -Contraction)	124
Derivation 4 — DPO from the KL-Constrained RLHF Objective	125

Part I

Foundations

MDPs, States, Actions & Rewards

Subscribe → aiengineeringinsider.substack.com/subscribe

The formalism every other chapter in this book builds on, and the first thing a sharp interviewer will make you state out loud.

1.1 Overview

Every reinforcement learning problem, however dressed up, reduces to the same object: a **Markov Decision Process** (MDP), the five-tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition function, $R(s, a, s')$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. An agent's job is to learn a **policy** $\pi(a | s)$, a mapping from states to actions (or to a distribution over actions), that maximizes expected cumulative discounted reward $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R_{t+1}]$.

The core concept that separates RL from supervised learning is the **closed loop**: the agent's actions change the data it later sees. There is no fixed dataset. A classifier that predicts the wrong label does not change tomorrow's images; an RL agent that takes a bad action moves into a region of the state space it then has to learn from. This single fact, that the data distribution is a function of the policy, is the source of nearly every difficulty in the field, and it is what the MDP formalism is designed to make precise (Sutton & Barto, 2018).

Markov Property

A state s_t is Markov if $P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_1, a_1, \dots, s_t, a_t)$. In plain language: the state must contain everything relevant to predicting the future. If it does not, you do not have an MDP, you have a partially observable problem wearing an MDP's clothes (a POMDP), and most of the convergence guarantees in this book stop applying.

1.1.1 The agent–environment loop

The components interact in a strict cycle: the agent observes state s_t , selects action $a_t \sim \pi(\cdot | s_t)$, and the environment returns a reward r_{t+1} and next state $s_{t+1} \sim P(\cdot | s_t, a_t)$. Figure 1.1 shows this loop together with the quantities each algorithm family in this book estimates.

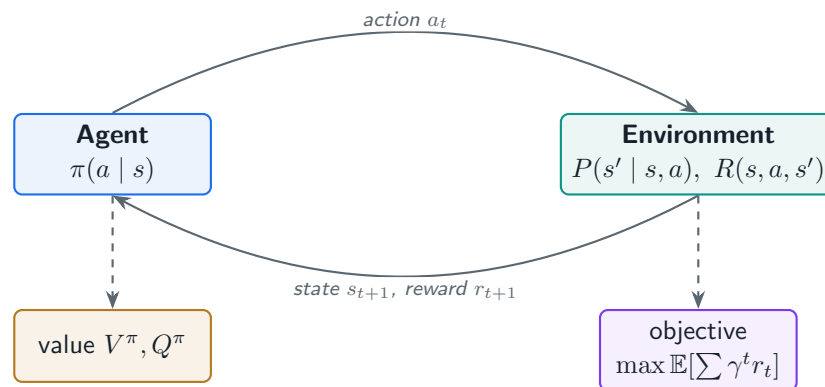


Figure 1.1: The MDP interaction loop. The agent influences the data it receives, which is the defining feature of reinforcement learning.

1.2 Why the Discount Factor Exists

The discount factor γ is not a minor implementation detail; it is what makes infinite-horizon problems mathematically tractable and what encodes a preference for sooner rewards over later, equally-sized ones. Without discounting, an infinite sequence of rewards may not converge to a finite sum, and value functions become ill-defined for continuing (non-episodic) tasks. The quantity $\frac{1}{1-\gamma}$ is the **effective horizon**: with $\gamma = 0.99$ the agent effectively plans about 100 steps ahead, and with $\gamma = 0.9$ only about 10.

★ Key Insight

When you are asked to “design an RL system” in an interview, stating the MDP formulation explicitly, state, action, reward, transition assumptions, episodic vs. continuing, before touching any algorithm, is the single highest-leverage thing you can do. It signals that you think about RL as problem formulation first, algorithm second.

1.3 Value Functions and the Bellman Equations

Two value functions recur throughout the book. The state-value $V^\pi(s)$ is the expected return from s following π ; the action-value $Q^\pi(s, a)$ is the expected return from taking a in s and following π thereafter. Both satisfy recursive **Bellman equations**, and the optimal versions satisfy the Bellman *optimality* equations with a max instead of an expectation over actions. The distinction, evaluation versus improvement, is the conceptual seam that Chapter 2 splits open.

1.4 State and Action Space Design

In production systems the gap between a *sufficient* (Markov) state and a *tractable* one is where most engineering effort goes. A fully sufficient representation is often too large to learn from efficiently, so practitioners compress, summarize, or use function approximation to project a richer state space into something learnable.

! Pitfall / Warning

A common interview trap: claiming that “more state information is always better.” In practice, overly large state spaces increase sample complexity and can make an otherwise solvable problem intractable. Production RL teams routinely caution that agents are sensitive to too many state inputs and recommend compression (e.g. autoencoding) when the natural state is large (see the case studies in Chapter 14).

1.5 The Project: a Reusable Tabular MDP

Every classical chapter in this book runs on one environment, `GridWorld`, defined in `rl_interview_lab/envs/gridworld.py`. It is a stochastic grid with a goal, a pit, walls, a per-step cost, and a “slip” probability that makes the dynamics genuinely random. Crucially it exposes *two* interfaces on the same dynamics: a *model* interface (`transitions`) returning the full $P(s', r | s, a)$ distribution for the planning methods of Chapter 2, and a *sampling* interface (`reset / step`) in the Gymnasium style for the model-free methods of Chapters 3–4.

Project Code — envs/gridworld.py

The model interface is what makes dynamic programming possible: the agent can “read the rules” instead of sampling them. The sampling interface is what every real agent is limited to. Building both on identical dynamics lets the book show *exactly* what changes when you move from planning to learning.

gridworld.py — the model interface used by dynamic programming

```
def transitions(self, state: State, action: Action) -> List[Transition]:
    """Full  $P(s', r | s, a)$  for one (state, action) pair."""
    if self.is_terminal(state):
        return [Transition(1.0, state, 0.0, True)]

    cell = self.decode(state)
    # The intended action plus the two perpendicular "slip" actions.
    perpendicular = {
        UP: (LEFT, RIGHT), DOWN: (LEFT, RIGHT),
        LEFT: (UP, DOWN), RIGHT: (UP, DOWN),
    }[action]
    outcomes = {action: 1.0 - self.slip_prob}
    for slip in perpendicular:
        outcomes[slip] = outcomes.get(slip, 0.0) + self.slip_prob / 2.0

    result = []
    for act, prob in outcomes.items():
        nxt = self.encode(self._move(cell, act))
        reward, done = self._reward(nxt)
        result.append(Transition(prob, nxt, reward, done))
    return result
```

The `slip_prob` is the whole point. With `slip_prob=0.1` the intended action succeeds 90% of

the time and “slips” to a perpendicular action 10% of the time. This makes the optimal policy avoid hugging the pit, because a slip near the edge of a cliff is expensive in expectation, exactly the kind of risk-aware behaviour that distinguishes a real policy from a memorised shortest path. In production you almost never have access to a clean `transitions` method like this; you have a simulator or logged data. The method exists here so Chapter 2 can compute the ground-truth optimal policy that the model-free chapters then approximate.

✓ Best Practice

Design the reward to express *what* you want, never *how* to achieve it. The small negative `step_reward` here (-0.04) encodes “finish quickly” without prescribing a route. Rewarding intermediate behaviours you *think* are good is the most common way to induce reward hacking (Chapter 11).

1.6 Comparison: MDP, POMDP, and the Bandit

Property	Multi-armed bandit	MDP	POMDP
State	none (single state)	fully observed	hidden; only observations
Actions affect future state	no	yes	yes
Credit assignment	immediate	delayed over time	delayed + perceptual aliasing
Core difficulty	exploration only	exploration + long-horizon credit	all of MDP + belief tracking
Typical solution	UCB, Thompson sampling	DP, TD, policy gradient	belief-state RL, recurrent policies

Table 1.1: The bandit is an MDP with one state; the POMDP is an MDP whose state you cannot see. Knowing which one you actually have is the first design decision.

1.7 Mock Interview Questions

Q1

Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation?

The Bellman equation for the state-value function under a policy π is an expectation:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')].$$

This describes the value of *following* a given policy. The Bellman *optimality* equation replaces the expectation over actions with a max:

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')].$$

The max is there because we are no longer averaging over a fixed policy's action distribution; we are asking what the *best possible* action is at each state, given that all future decisions are also optimal. That makes the optimality equation a fixed-point equation for the optimal value function. An interviewer asking this is testing whether you understand that policy evaluation (expectation) and policy improvement (max) are genuinely different operations, not interchangeable notation.

Q2

How does the discount factor γ affect agent behaviour, and what happens as $\gamma \rightarrow 1$?

A small γ produces myopic agents that overweight immediate reward; a γ close to 1 produces farsighted agents that tolerate short-term loss for long-term gain, but at the cost of higher variance in value estimates and slower convergence, since the effective horizon $\frac{1}{1-\gamma}$ grows without bound. As $\gamma \rightarrow 1$ in a continuing task, $V^\pi(s)$ can diverge unless rewards are bounded and the induced Markov chain is well-behaved (e.g. ergodic). In practice γ is a hyperparameter that encodes the business's actual planning horizon: a notification system optimizing weekly engagement should not use the same γ as a high-frequency trading agent.

Q3

What is the Markov property, and what do you do when your state is not Markov?

The Markov property says the current state is a sufficient statistic for the future: conditioning on the full history adds nothing. When it fails, you have a POMDP. The standard fixes, in increasing order of cost: (1) *augment* the state with the missing information (e.g. stack the last k frames so velocity is observable, as the Atari DQN does); (2) maintain a *belief state*, a posterior over hidden state, and treat that as the MDP state; or (3) use a *recurrent* or transformer policy that learns its own summary of history. The interview signal is recognising that “my agent behaves erratically” is often a non-Markov state problem, not an algorithm problem.

Q4

Distinguish V^π , Q^π , V^* , and Q^* . Which do you actually need to act?

$V^\pi(s)$ is expected return from s under π ; $Q^\pi(s, a)$ conditions additionally on the first action. The starred versions are the optima. To *act* greedily you need Q , not V : $\arg \max_a Q^*(s, a)$ requires no model, whereas $\arg \max_a \sum_{s'} P(s' | s, a)[r + \gamma V^*(s')]$ needs the transition model. This is the entire reason model-free control methods learn Q : it makes the greedy step model-free. Stating this crisply is a strong signal because it explains a design choice that runs through Chapters 3–5.

Q5

A product manager wants to “use RL” for a problem with no sequential structure. How do you respond?

I would first check whether actions actually influence future state. If they do not, it is a contextual bandit, not a full MDP, and bandit methods (Thompson sampling, LinUCB) are simpler, more sample-efficient, and far easier to evaluate offline. Reaching for full RL

when a bandit suffices is a classic over-engineering failure: you inherit credit-assignment and stability problems you did not need. The highest-value answer demonstrates judgment about *when not to use RL*, which interviewers weight heavily.

Q6

How would you formulate “reduce customer churn” as an MDP? Walk through each element.

State: customer features plus engagement history (recency, frequency, tenure, recent support contacts). **Action:** which intervention to send (discount, email, no-op). **Reward:** a sparse retention signal at the horizon, often shaped with a small per-step cost for expensive interventions. **Transition:** stochastic and unknown, so this is model-free and likely offline. γ : tied to the business horizon (monthly vs. annual value). The hard parts are that the reward is delayed and the data is observational, which pushes toward offline RL and off-policy evaluation rather than online exploration on real customers.

Q7

Why can a small change to the reward function completely change the optimal policy, and why is that dangerous?

Because the optimal policy is defined entirely by the reward: it is $\arg \max$ over expected discounted reward. Two reward functions that look similar can induce wildly different optima if their relative magnitudes shift the trade-off between sub-goals. This is dangerous because reward is usually hand-designed, and the agent will exploit *exactly* what you wrote, not what you meant, the boat-racing agent that loops collecting power-ups instead of finishing the race is the canonical example. The mitigation is potential-based reward shaping (Ng et al., 1999), which provably preserves the optimal policy while adding guidance.

Q8

In our GridWorld the dynamics are stochastic (a 10% slip). How does that change the optimal policy versus the deterministic case?

In the deterministic case the optimal policy is the shortest path, and it can safely run adjacent to the pit. Add a slip probability and the same path now has a real chance of slipping *into* the pit, whose large negative reward dominates the expectation. The optimal policy therefore detours away from the cliff edge, accepting a longer expected path to reduce catastrophic-outcome risk. This is risk sensitivity emerging purely from expectation maximization under stochastic dynamics, and it is exactly what makes the value-iteration solution in Chapter 2 more interesting than a graph shortest-path.

Q9

What is the difference between episodic and continuing tasks, and how does each handle the return?

Episodic tasks have terminal states (a game ends, an episode resets); the return is a finite sum to termination, so $\gamma = 1$ is sometimes safe. Continuing tasks run forever; the return must be discounted ($\gamma < 1$) to stay finite, or reformulated with the average-reward criterion. The practical trap appears with time-limit truncation: when an episode is cut off by a step

limit rather than a true terminal, you must *not* zero out the bootstrap term, because the task did not actually end. Our DQN and PPO code in Chapters 5 and 8 handle this by storing `terminated` (true MDP terminal) separately from `truncated` (time limit).

Q10

Your state space is huge and continuous. What are your options, and what do you lose with each?

Options: (1) *discretization* or tile coding, simple but suffers the curse of dimensionality and loses smoothness across cell boundaries; (2) *linear function approximation* over hand-engineered features, stable and analyzable but limited by feature quality; (3) *deep function approximation*, expressive and the modern default, but it forfeits convergence guarantees and exposes you to the deadly triad (Chapter 5). The judgment being tested is that there is no free lunch: every step up in representational power trades away a guarantee or adds an instability you now have to manage.

References

- Bellman, R. (1957). *Dynamic programming*. Princeton University Press.
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning* (pp. 278–287). Morgan Kaufmann.
- Puterman, M. L. (1994). *Markov decision processes: Discrete stochastic dynamic programming*. Wiley.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Dynamic Programming

Subscribe → aiengineeringinsider.substack.com/subscribe

Policy iteration, value iteration, and the convergence guarantees that deep RL gives up.

2.1 Overview

Dynamic programming (DP) methods assume full knowledge of the MDP, the transition function P and reward function R are known, and compute exact value functions and optimal policies through repeated application of the Bellman equations. No real-world system has a fully known model, so DP is never directly deployed in production, yet it is the conceptual root of nearly everything that follows: TD learning, Q-learning, and policy gradients are all approximations of DP operations when the model is unknown (Sutton & Barto, 2018).

The core concept is **bootstrapping with a known model**: DP estimates a state’s value from the estimated values of its successors, weighted by the known transition probabilities. Two ideas drive the whole chapter. **Policy evaluation** computes V^π for a fixed policy by iterating the Bellman expectation equation to convergence. **Policy improvement** constructs a weakly better policy by acting greedily with respect to V^π . The **policy improvement theorem** guarantees the new policy is at least as good, and strictly better unless both are already optimal, which is what makes the alternation converge.

Generalized Policy Iteration (GPI)

Almost every RL algorithm is an instance of GPI: maintain a value estimate and a policy, repeatedly make the value more consistent with the policy (evaluation) and the policy greedier with respect to the value (improvement). DP does both exactly; Monte Carlo and TD do both approximately from samples. Seeing later algorithms as “GPI with one piece approximated” is the mental model interviewers reward.

2.2 Policy Iteration vs. Value Iteration

Policy iteration runs evaluation to convergence before each improvement step. **Value iteration** truncates evaluation to a single sweep and folds improvement in via the Bellman optimality backup, $V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R + \gamma V_k(s')]$. They reach the same optimum; the choice is about cost per iteration versus number of iterations. Figure 2.1 shows both loops.

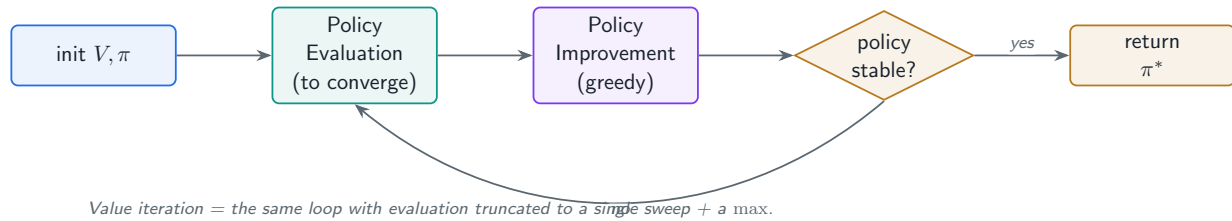


Figure 2.1: Policy iteration as generalized policy iteration. Value iteration collapses the inner evaluation loop into one optimality backup.

★ Key Insight

The reason DP does not scale: both methods sweep the entire state space every iteration and require P and R exactly. Neither holds in any state space large enough to be interesting. Every later chapter is, in some sense, an answer to “how do we approximate DP when we lack a model and cannot enumerate states.”

2.3 Bellman Optimality as a Contraction

The Bellman optimality operator T is a γ -contraction in the max-norm: $\|TV - TV'\|_\infty \leq \gamma\|V - V'\|_\infty$. By the Banach fixed-point theorem, repeated application converges geometrically to the unique fixed point V^* , at rate γ . This is the theoretical bedrock under TD learning’s convergence proofs, and worth stating precisely if a research-track interview pushes into theory.

2.4 The Project: Planning on GridWorld

`rl_interview_lab/classical/dynamic_programming.py` implements both algorithms against the `transitions` model from Chapter 1. On the default 4×4 GridWorld, policy iteration converges in 3 policy updates and value iteration in 15 sweeps, to the *same* policy, a concrete demonstration of the cost trade-off.

dynamic_programming.py — the value-iteration backup

```

def value_iteration(env, gamma=0.95, theta=1e-8):
    """Collapse evaluation + improvement into one Bellman optimality backup."""
    V = np.zeros(env.n_states)
    for iteration in range(1, 100000):
        delta = 0.0
        for s in range(env.n_states):
            if env.is_terminal(s):
                continue
            # One-step look-ahead over every action, then take the max.
            q_sa = [
                sum(t.prob * (t.reward + gamma * V[t.next_state]))
                for t in env.transitions(s, a)
                for a in range(env.n_actions)
            ]
            v_new = max(q_sa)  # Bellman OPTIMALITY backup
  
```